Práctica 4:

Mi Agente en la Nube

Desplegar el contenedor del Agente de Soporte en Railway

Objetivo

En la Práctica 3 el agente quedó empaquetado en un contenedor Docker, pero ese contenedor corre en la computadora del alumno: si la laptop se apaga, se suspende o se queda sin internet, el agente deja de atender quejas. En esta práctica el alumno llevará **ese mismo contenedor** a un servidor en la nube, usando **Railway**, para que el agente funcione las 24 horas sin depender de ninguna computadora personal.

Esta práctica enseña el último paso del ciclo de vida de un programa: el **despliegue** (*deploy*). Al terminarla, el alumno sabrá crear un proyecto en una plataforma de nube, configurar los secretos como variables del servicio, desplegar desde un Dockerfile, leer los registros de un servidor remoto, actualizar el agente, vigilar el consumo de crédito y apagar los recursos cuando ya no se necesitan.

¿Qué es Railway y para qué sirve?

Railway (railway.com) es una **plataforma como servicio** (PaaS, por sus siglas en inglés). El alumno le entrega su código con un Dockerfile, y Railway hace todo lo demás en sus propios servidores: construye la imagen, crea el contenedor, lo mantiene encendido, lo vuelve a arrancar si falla y guarda sus registros. No hay que rentar ni configurar un servidor Linux, instalar Docker en él ni preocuparse por actualizaciones del sistema operativo.

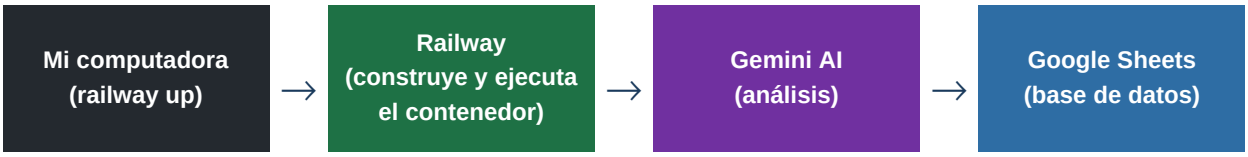
Dicho de otra forma: Railway es como tener el Docker Desktop de la Práctica 3, pero en una computadora que nunca se apaga y que no es la del alumno. Los mismos conceptos (imagen, contenedor, variables de entorno, registros, reinicio automático) aparecen con otros nombres en su panel web. Otras plataformas parecidas son Render, Fly.io, Heroku y Google Cloud Run; todas aceptan un Dockerfile.

Opción	¿El agente sigue si apago mi laptop?	Revisa la hoja cada…	Costo aproximado
Práctica 1: script en la laptop	No	1 minuto	Gratis
Práctica 2: GitHub Actions	Sí	30 minutos	Gratis (2,000 min/mes en repo privado)
Práctica 3: Docker en la laptop	No	1 minuto	Gratis
Práctica 4: Railway	**Sí**	**1 minuto**	Crédito de prueba; después desde $1/mes de crédito gratis o $5/mes

Railway combina lo mejor de las prácticas anteriores: funciona con la computadora apagada (como GitHub Actions) y responde en un minuto (como el modo continuo de las Prácticas 1 y 3). Se eligió Railway sobre Render porque, en octubre de 2026, el plan gratuito de Render solo cubre servicios web que se "duermen" tras 15 minutos sin visitas, y un proceso en segundo plano como el agente cuesta desde 7 dólares al mes. Railway, en cambio, ofrece una prueba

gratuita sin tarjeta.

Funcionamiento del Agente en Railway



Ni el script ni el Dockerfile cambian. La computadora del alumno solo se usa para **enviar** el proyecto a Railway; a partir de ahí, la construcción de la imagen y la ejecución del agente ocurren en los servidores de Railway. Los secretos se guardan en Railway como variables del servicio, igual que se guardaron como Secrets en GitHub en la Práctica 2.

Requisitos Previos

- Haber terminado la **Práctica 3**: el Dockerfile y el .dockerignore deben funcionar (el contenedor ya procesó quejas en la computadora).
- El archivo **.gitignore** debe excluir *.env*, *token.env* y *client_secrets.json*. En esta práctica es más importante que nunca (ver Fase 4).
- Tener el archivo **authorized_user.json** vigente en *%APPDATA%\gspread*.
- Una cuenta de **GitHub** (la misma de la Práctica 2) para registrarse en Railway.

Herramienta	Para qué sirve	Costo
Cuenta de Railway	La plataforma donde correrá el agente	Prueba gratuita sin tarjeta
Node.js (LTS)	Necesario para instalar la herramienta de terminal de Railway	Gratuito
Railway CLI	Comandos *railway* para crear, desplegar y ver registros	Gratuito
Proyecto de la Práctica 3	Script, requirements.txt, Dockerfile y .dockerignore	—

Conceptos Clave

Concepto	Qué es
PaaS	*Platform as a Service*: servicio de nube donde el usuario entrega su código y la plataforma se encarga de los servidores, el sistema operativo y la ejecución.
Despliegue (deploy)	Poner una versión del programa a funcionar en el servidor. Cada vez que se envía una versión nueva se crea un *deployment* nuevo.
Proyecto	Contenedor lógico en Railway que agrupa servicios relacionados. Aquí: *agente-soporte-ia*.
Servicio	Un programa que corre dentro del proyecto. Aquí: *agente*. Equivale al contenedor de la Práctica 3.
Build	La construcción de la imagen a partir del Dockerfile. Railway la hace en sus servidores; es el equivalente a *docker build*.
Variables del servicio	Variables de entorno guardadas en Railway que el contenedor recibe al arrancar. Equivalen a los Secrets de GitHub y a *--env-file*.
Crédito / uso	Railway cobra por segundo según la memoria y el procesador que usa el servicio, y lo descuenta de un crédito mensual.
CLI	*Command Line Interface*: herramienta para controlar Railway desde la terminal.

FASE 1: Crear la Cuenta e Instalar la Herramienta de Railway

Paso 1.1 — Crear la Cuenta de Railway

- Entrar a **railway.com** y elegir **Sign in** → **Continue with GitHub**, con la misma cuenta de GitHub de la Práctica 2. Autorizar el acceso que solicita Railway.
- Aceptar los términos. La cuenta nueva arranca en la **prueba gratuita**: un crédito único de 5 dólares que vence a los 30 días y **no pide tarjeta**.

Importante: No es necesario registrar una tarjeta para esta práctica. Si en algún momento Railway la pide, consultar con el profesor antes de capturarla. Al terminar la práctica se explicará cómo apagar el servicio para no consumir crédito (Paso 5.4).

Paso 1.2 — Instalar Node.js

La herramienta de terminal de Railway se instala con **npm**, el gestor de paquetes de Node.js. Primero revisar si Node.js ya está instalado:

```
node --version
```

Si muestra una versión (por ejemplo *v24.x.x*), continuar con el Paso 1.3. Si el comando no se reconoce, instalar Node.js con el siguiente comando (o descargando la versión LTS desde **nodejs.org**) y después **abrir una terminal nueva**:

```
winget install OpenJS.NodeJS.LTS
```

Paso 1.3 — Instalar la Railway CLI

En la terminal, ejecutar:

```
npm install -g @railway/cli
railway --version
```

La opción *-g* instala la herramienta de forma global, para usar el comando *railway* desde cualquier carpeta. El segundo comando debe mostrar la versión instalada (por ejemplo *railway 5.71.0*).

Nota: Si PowerShell responde *la ejecución de scripts está deshabilitada en este sistema*, ejecutar una sola vez *Set-ExecutionPolicy -Scope CurrentUser RemoteSigned*, confirmar con *S* y repetir el comando. Este cambio solo afecta al usuario actual.

Paso 1.4 — Iniciar Sesión desde la Terminal

```
railway login
railway whoami
```

El primer comando abre el navegador para autorizar la terminal con la cuenta de Railway. Al aceptar, la terminal muestra **Signed in as** seguido del correo de la cuenta. El segundo comando confirma con qué cuenta quedó la sesión.

FASE 2: Crear el Proyecto y el Servicio

Paso 2.1 — Crear el Proyecto

En la terminal, **dentro de la carpeta del proyecto** del agente, ejecutar:

```
railway init --name agente-soporte-ia
```

Railway crea el proyecto y **vincula** la carpeta actual con él: a partir de ahora, los comandos *railway* que se ejecuten en esta carpeta se aplican a ese proyecto. Si pregunta en qué *workspace* crearlo, elegir el que tiene el nombre de la cuenta.

Paso 2.2 — Crear el Servicio

```
railway add --service agente
```

- Cuando pregunte *What do you need?*, elegir **Empty Service** con las flechas y Enter.
- Si pide variables, dejarlo vacío y presionar Enter: las variables se configuran en la Fase 3.

Para verlo de forma visual, entrar a **railway.com/dashboard**: debe aparecer el proyecto **agente-soporte-ia** y, dentro, un recuadro con el servicio **agente** todavía sin desplegar.

Concepto: El servicio está vacío porque aún no se le ha enviado código. Es como haber creado un contenedor con nombre pero sin imagen. En la Fase 4 se le enviará el proyecto y Railway construirá la imagen.

FASE 3: Configurar los Secretos como Variables del Servicio

El agente necesita los mismos dos secretos de siempre: **GEMINI_API_KEY** y **GOOGLE_TOKEN_JSON**. En la Práctica 2 vivían en los Secrets de GitHub y en la Práctica 3 en los archivos .env y token.env. Ahora se guardarán en Railway, que los entregará al contenedor como variables de entorno cada vez que arranque.

Paso 3.1 — Abrir la Pestaña de Variables

- En **railway.com/dashboard**, abrir el proyecto **agente-soporte-ia** y hacer clic en el servicio **agente**.
- Entrar a la pestaña **Variables**.

Paso 3.2 — Agregar las Dos Variables

Con el botón **New Variable**, agregar una por una:

Nombre de la variable	Valor que se pega
GEMINI_API_KEY	La API Key de Gemini: lo que está después del signo igual en el archivo .env.
GOOGLE_TOKEN_JSON	Todo el contenido de *%APPDATA%\gspread\authorized_user.json*, abierto con el Bloc de notas: desde la llave **{** del inicio hasta la llave **}** del final. Es lo mismo que va después del signo igual en token.env.

- Revisar que los nombres estén escritos exactamente así, en mayúsculas y con guion bajo.
- Si el panel muestra un aviso de cambios pendientes, aplicarlos con el botón **Deploy** o **Apply**. Mientras el servicio no tenga código, no pasa nada más.

Importante: Pegar los valores **sin comillas** alrededor. Además, no cargar el token con una tubería de PowerShell como *Get-Content ... | railway variable set*: Windows PowerShell 5.1 le agrega al inicio unos caracteres invisibles (BOM) y el script no puede leer el JSON. El panel web, en cambio, guarda el texto tal cual se pega.

Nota: El panel oculta los valores con asteriscos. No tomar capturas con los valores visibles: quien tenga el token puede leer y modificar las hojas de cálculo del alumno.

FASE 4: Desplegar el Agente

Paso 4.1 — Revisar Qué Se Va a Enviar

El comando *railway up* envía a Railway **toda la carpeta del proyecto**, excepto lo que esté en el archivo **.gitignore**. Antes de desplegar, abrir el .gitignore y confirmar que contiene, al menos:

```
.env
token.env
client_secrets.json
```

Concepto: Hay dos filtros. El **.gitignore** decide qué archivos salen de la computadora hacia Railway, y el **.dockerignore** decide qué archivos entran a la imagen. Aunque el Dockerfile solo copie dos archivos, un secreto que no esté en el .gitignore viajaría hasta los servidores de Railway.

Paso 4.2 — Ejecutar railway up

Dentro de la carpeta del proyecto, ejecutar:

```
railway up --service agente
```

Railway sube la carpeta, detecta el **Dockerfile** y construye la imagen en sus servidores. La terminal muestra los mismos pasos que *docker build* en la Práctica 3: *FROM python:3.12-slim*, la instalación de librerías con pip y la copia del script. La primera vez tarda uno o dos minutos. Al terminar aparece **Deploy complete** y, a continuación, los mensajes del agente arrancando.

Para salir de la vista de registros, presionar Ctrl+C. Esto solo cierra la vista: el agente **sigue corriendo** en Railway.

Concepto: No fue necesario subir la imagen a Docker Hub ni a ningún registro: Railway recibe el Dockerfile y construye la imagen por su cuenta. Tampoco hizo falta agregar *--una-vez*: sin argumentos, el ENTRYPOINT arranca el modo continuo, que es justo lo que se quiere en un servidor.

Paso 4.3 — Verificar el Estado y los Registros

```
railway status
railway logs --service agente
```

- **railway status** muestra el proyecto, el servicio y su estado. Debe decir **status: Online**.
- **railway logs** muestra lo que el agente ha impreso: el cuadro de inicio *Agente de Soporte - Iniciando...* y, cada minuto, un bloque con las filas ya procesadas y las nuevas.
- En el panel web, el recuadro del servicio se ve en verde y la pestaña **Deployments** muestra el despliegue como *Active*. Desde ese despliegue también se pueden consultar los registros.

Nota: El cuadro de inicio dice *Presiona Ctrl+C para detener*. En Railway ese mensaje no aplica, porque no hay una terminal conectada al agente; para detenerlo se usa el Paso 5.4.

FASE 5: Probar, Actualizar y Administrar el Agente en la Nube

Paso 5.1 — Pausar GitHub Actions

El workflow de la Práctica 2 y el agente en Railway leen la misma hoja. Si ambos están activos, pueden tomar la misma fila al mismo tiempo y escribirla dos veces. Para que solo procese Railway, pausar el workflow: en GitHub, pestaña **Actions** → *Agente de Soporte* → botón **"..."** → **Disable workflow**. También se puede hacer desde la terminal con:

```
gh workflow disable "Agente de Soporte"
```

El archivo del workflow se queda en el repositorio; solo deja de ejecutarse. Si el contenedor de la Práctica 3 sigue corriendo en la computadora, detenerlo también con *docker stop agente*.

Paso 5.2 — Prueba de Punta a Punta con la Laptop Apagada

- Enviar una queja nueva desde el Google Form.
- **Apagar o suspender la computadora** (o cerrar la terminal y desconectarse del Wi-Fi) y esperar dos minutos.

- Desde el celular, abrir el Google Sheet: las columnas D, E y F de la fila nueva deben estar llenas, aunque la computadora esté apagada.
- Al volver a encender la computadora, revisar *railway logs --service agente*: debe aparecer la fila con el nombre del cliente, la clasificación, el sentimiento y la línea **[OK] Resultado escrito en la fila N**.

Concepto: Esta es la diferencia que justifica la práctica: en las Prácticas 1 y 3 la fila se habría quedado sin procesar hasta volver a encender la laptop, y en la Práctica 2 habría esperado hasta 30 minutos. Si Gemini responde *503 UNAVAILABLE*, la fila se reintenta sola en el siguiente ciclo, igual que en la Práctica 3.

Paso 5.3 — Actualizar el Agente

Si se modifica el script (por ejemplo, para cambiar las categorías de clasificación), Railway no se entera solo. Hay que volver a enviar el proyecto:

```
railway up --service agente
```

Railway construye una imagen nueva (reutilizando en caché la capa de *pip install* si *requirements.txt* no cambió), arranca el contenedor nuevo y después apaga el anterior. En la pestaña **Deployments** queda el historial de versiones.

Nota: Si se renueva el token de Google (por ejemplo, tras un error *invalid_grant*), basta con editar la variable **GOOGLE_TOKEN_JSON** en la pestaña Variables y aplicar el cambio: Railway vuelve a desplegar el servicio con el valor nuevo. Recordar actualizar también el secret de GitHub si se piensa reactivar el workflow.

Paso 5.4 — Vigilar el Consumo y Apagar el Servicio

En la nube, un programa encendido cuesta dinero aunque no esté haciendo nada. Railway cobra por segundo según la memoria y el procesador que usa el servicio:

Plan (octubre de 2026)	Qué incluye
Prueba gratuita	Crédito único de $5 que vence a los 30 días. No pide tarjeta.
Free	$1 de crédito cada mes, hasta 0.5 GB de memoria por servicio.
Hobby	$5 al mes, que incluyen $5 de consumo.
Precio del uso	Aproximadamente $10 por GB de memoria y $20 por procesador (vCPU) al mes, cobrados por segundo.

- Para ver el consumo, en el panel web abrir el menú de la cuenta → **Usage**. Ahí aparece cuánto crédito se ha usado y una estimación del mes.
- Para **apagar** el agente sin borrar nada, ejecutar el comando de abajo. El servicio queda *Offline* y deja de consumir crédito. Para volver a encenderlo, basta con *railway up*.

```
railway down --service agente
```

Importante: Al terminar la práctica, decidir con el profesor si el agente se queda en Railway o se apaga. Si se apaga, **volver a activar el workflow** de GitHub Actions (**Enable workflow** o *gh workflow enable "Agente de Soporte"*) para que las quejas se sigan procesando. Siempre debe haber **uno y solo uno** procesando la hoja.

Solución de Errores Comunes

Error	Causa probable	Solución
npm o node no se reconoce como nombre de un cmdlet	Node.js no está instalado o la terminal se abrió antes de instalarlo	Instalar Node.js (Paso 1.2) y abrir una terminal nueva
la ejecución de scripts está deshabilitada en este sistema	Política de ejecución de PowerShell	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned (Paso 1.3)
Unauthorized. Please login with railway login	No se ha iniciado sesión o la sesión caducó	Ejecutar railway login
Error de proyecto no vinculado (*No linked project*)	El comando se ejecutó fuera de la carpeta del proyecto	Entrar a la carpeta del agente; si se creó en otra, ejecutar railway link
El servicio no se encuentra	El nombre del servicio está mal escrito	Revisar con railway status y usar el mismo nombre en --service
El build falla en la instalación de librerías	Error en requirements.txt	Probar primero docker build en la computadora (Práctica 3) y corregir
"Debes reemplazar GEMINI_API_KEY" en los registros	Falta la variable o su nombre está mal escrito	Revisar la pestaña Variables (Paso 3.2)
"No se encontró el archivo client_secrets.json" en los registros	Falta GOOGLE_TOKEN_JSON o tiene otro nombre	Agregar la variable con el nombre exacto
JSONDecodeError al leer GOOGLE_TOKEN_JSON	El valor quedó incompleto, con comillas extra o con BOM (tubería de PowerShell)	Volver a pegar el contenido completo del archivo en el panel web, de { a }
invalid_grant en los registros	El token caducó o fue revocado	Generar un token nuevo y actualizar la variable (Paso 5.3)
Una queja se procesó dos veces	El workflow de GitHub Actions o el contenedor local siguen activos	Dejar solo uno procesando (Paso 5.1)
503 UNAVAILABLE ... high demand	Gemini está saturado en ese momento	Nada: la fila se reintenta sola en el siguiente ciclo
El servicio aparece como Offline o en pausa	Se ejecutó railway down o se agotó el crédito	Revisar Usage; volver a desplegar con railway up

Comparación: Prácticas 2, 3 y 4

Aspecto	Práctica 2 (GitHub Actions)	Práctica 3 (Docker local)	Práctica 4 (Railway)
¿Dónde corre?	En un runner de GitHub	En la computadora del alumno	En los servidores de Railway
¿Quién construye la imagen?	No hay imagen	docker build	Railway, desde el Dockerfile
¿Quién lo repite?	El cron del workflow (30 min)	schedule en el contenedor (1 min)	schedule en el contenedor (1 min)
Secretos	Secrets del repositorio	--env-file .env y token.env	Pestaña Variables del servicio
¿Dónde se ven los mensajes?	Pestaña Actions	docker logs agente	railway logs o panel web
¿Funciona con la laptop apagada?	Sí	No	Sí
¿Cómo se actualiza?	git push	docker build + crear contenedor	railway up

¿Y después? Railway también puede conectarse directamente a un repositorio de GitHub para desplegar solo cada vez que se hace *git push* (despliegue continuo), y puede ejecutar el agente como tarea programada (*Cron Schedule* en la configuración del servicio, con *--una-vez*) en lugar del modo continuo. Son dos formas de profundizar lo aprendido en esta práctica.

Entregables

- **Captura 1:** la terminal con *railway --version* y *railway whoami* (Pasos 1.3 y 1.4).
- **Captura 2:** la pestaña Variables con las dos variables **ocultas** con asteriscos (Paso 3.2).
- **Captura 3:** el final de *railway up* con **Deploy complete** y el arranque del agente (Paso 4.2).
- **Captura 4:** el panel de Railway con el proyecto y el servicio **agente** en verde (Paso 4.3).
- **Captura 5:** los registros con la queja procesada en la prueba con la laptop apagada, y el Google Sheet con esa fila clasificada (Paso 5.2).
- **Captura 6:** la página Usage con el consumo del servicio (Paso 5.4).
- **Respuestas** a las preguntas de reflexión.

Preguntas de Reflexión

- Explicar con palabras propias qué es una PaaS y en qué se diferencia de rentar un servidor (VPS) e instalar Docker en él.
- ¿Por qué no fue necesario cambiar ni una línea del script ni del Dockerfile para llevar el agente de la laptop a Railway?
- ¿Qué pasaría si el archivo .env no estuviera en el .gitignore al ejecutar *railway up*? ¿Lo evitaría el .dockerignore?
- ¿Por qué no deben estar activos al mismo tiempo GitHub Actions y Railway? ¿Qué cambio en el script permitiría que dos agentes trabajaran sobre la misma hoja sin pisarse?

- Con los precios del Paso 5.4 y el consumo observado en Usage, estimar cuánto costaría mantener el agente un mes. ¿Alcanza el plan Free?
- Para este agente, ¿qué opción recomendarías a una pequeña empresa real: GitHub Actions, Railway o un servidor propio? Justificar con costo, tiempo de respuesta y mantenimiento.